

Salon Management System — Technical Build Spec

Companion to: Salon-Management-System-Implementation-Guide.md **Purpose:** Close the gaps between scoping and coding. This is for the developer's desk, not the client meeting.

1. Roles — Defined First, Build This Before Anything Else

Roles are step zero. Every screen, every API endpoint, every menu item downstream depends on this being locked first — build the `Role/Permission/RolePermission` tables and seed these 4 roles before writing any feature code.

Role	Who	Access Level
Owner/CEO	Client herself	Full access to every module + User Management (create users, assign roles, override individual permissions)
Manager	Second-in-command / senior staff	Full operational access (Bookings, Billing, CRM, Inventory, Reports, Payroll-view) — NO User Management, cannot finalize Payroll
Stylist	Hair/beauty service staff	Own attendance punch, own booking calendar (view only), own commission/earnings view — nothing else
Massage/Spa Therapist	Spa/massage service staff	Same access as Stylist — kept as a separate role (not a Stylist sub-type) since service specialization and commission slabs commonly differ between hair/beauty and spa/massage; splitting the role now avoids a messy override system later.

Structurally, Stylist and Massage Therapist will share the same `RolePermission` set (identical menu/access) — they're kept as two rows in the `Role` table rather than one, purely so `StaffProfile.specialization` and `CommissionSlab` assignment stay clean per role without needing exception logic. If in practice they end up needing different screen access later, the separation is already there for free.

Add a `Receptionist` role later (Phase 2, if client hires front-desk staff distinct from Manager) — not seeded now since client hasn't confirmed she has one.

2. Data Model (Core Entities + Relationships)

1.1 Entity List

Branch (id, name, address, phone, is_active)
[single row today, schema supports multiple]

User (id, name, phone, email, password_hash, role_id, branch_id, is_active, created_by, created_at)

Role (id, name, description)
[seed: Owner/CEO, Manager, Receptionist, Stylist/Staff, Accountant]

Permission (id, module, action)
[e.g. module='billing', action='create' | 'view' | 'edit' | 'delete' | 'approve']

RolePermission (id, role_id, permission_id)
[many-to-many – this is what makes menus dynamic per role]

UserMenuOverride (id, user_id, permission_id, granted boolean)
[OPTIONAL layer – lets CEO override a specific user's access beyond their role,
e.g. give one receptionist access to Reports without making her a Manager]

StaffProfile (id, user_id, designation, specialization[], commission_slab_id, base_salary, shift_id, joining_date)

CommissionSlab (id, name, type[flat|tiered], rules_json)
[rules_json holds the actual slab logic – see Section 3]

ServiceCategory (id, name)

ServiceMaster (id, category_id, name, duration_minutes, price, commission_slab_override_id NULLABLE, is_active)

ProductMaster (id, name, sku, unit, purchase_price, sale_price, current_stock, reorder_level, is_active)

Customer (id, name, phone [unique], dob, anniversary_date, gender, tags[], preferred_stylist_id NULLABLE, notes, created_at)

PackageMaster (id, name, type[prepaid_bundle|membership], validity_days, price, included_services json, credit_count, discount_logic_json)
[see Section 4 for how discount_logic_json is structured]

CustomerPackage (id, customer_id, package_master_id, purchase_date, expiry_date, credits_remaining, status[active|expired|exhausted], invoice_id)

Booking (id, customer_id, branch_id, stylist_id, service_ids[], booking_date, start_time, end_time, status[booked|confirmed|in_progress|completed|no_show|cancelled], source[internal|online], created_by, notes)

Invoice (id, customer_id, branch_id, invoice_date, subtotal, tax_amount, discount_amount, total_amount, payment_mode[cash|upi|card|split], payment_status, created_by)

InvoiceLineItem (id, invoice_id, item_type[service|product], item_id, staff_id NULLABLE for products, quantity, unit_price, discount, tax, line_total, package_redemption_id NULLABLE)
[staff_id here is what drives commission – must be set per line, not per

```
invoice,
    since one invoice can span multiple stylists]

CommissionEntry (id, staff_id, invoice_line_item_id, commission_amount,
calculated_at, payroll_run_id NULLABLE)

Attendance (id, staff_id, date, punch_in_time, punch_out_time, status[present|
absent|half_day|leave], remarks)

PayrollRun (id, month, year, generated_by, status[draft|finalized|paid])
PayrollEntry (id, payroll_run_id, staff_id, base_salary, commission_total,
deductions, net_payable)

WhatsAppTemplate (id, name, trigger_type[birthday|anniversary|festival|
package_expiry|appointment_reminder|winback], message_body, is_active)
WhatsAppCampaignLog (id, template_id, customer_id, sent_at, status[sent|
delivered|read|failed])
```

1.2 Key Relationships to Get Right

- `InvoiceLineItem.staff_id` is per-line, not per-invoice — one billing transaction can involve 2 stylists (e.g. haircut by A, spa by B) and each needs separate commission attribution.
 - `CustomerPackage.credits_remaining` decrements via `InvoiceLineItem.package_redemption_id` — never delete/hard-decrement without an audit trail; log every redemption against the invoice line that consumed it.
 - `RolePermission` + `UserMenuOverride` together determine the rendered menu — see RBAC section, this is not optional plumbing, it's core to the "single login, different menu" requirement.
-

3. Screen List & Field-Level Spec (Phase 1 — Internal Only)

For each screen: purpose, fields, and the one or two business rules that aren't obvious from the field list alone.

2.1 Login Screen

- Fields: Phone/Email, Password
- Single login URL for all roles. Post-login, redirect + render menu based on resolved permissions (see RBAC).

2.2 Dashboard (role-dependent content — see RBAC for what each role sees)

- Owner/CEO view: full KPI set (Section on Dashboard in main guide) + User Management access
- Receptionist view: today's bookings queue + quick billing shortcut only

2.3 Booking Screen (Internal)

- Fields: Customer (search-or-create inline), Service(s) [multi-select], Stylist (filtered to those who can perform selected service, per StaffProfile.specialization), Date, Time slot, Notes
- Rule: on stylist+date+time selection, run conflict check — reject if stylist already has an overlapping booking. Show next available slot for that stylist as a suggestion, don't just block.
- Rule: walk-ins are bookings with `start_time = now`, no separate walk-in entity — keeps reporting simpler (one funnel, not two data sources).

2.4 Billing/POS Screen

- Fields: Customer (auto-pull from active booking if invoicing against one, else search-or-create), line items (add service or product), per-line staff assignment, apply package redemption toggle (if customer has active package, show available credits inline), payment mode, split payment amounts if applicable
- Rule: if item is a product, reduce `ProductMaster.current_stock` on save; block save if stock insufficient (with an override permission for Manager+ role to force it, logged).
- Rule: package redemption reduces price to ₹0 (or membership discount %) for that line and logs `package_redemption_id` — does not just apply a manual discount, must trace to the CustomerPackage record.

2.5 Package Sale Screen

- Fields: Customer, Package (from PackageMaster), payment
- On save: create CustomerPackage with computed expiry_date and credits_remaining seeded from PackageMaster.credit_count

2.6 Attendance Screen (Punch)

- Fields: Staff selects self (or admin punches on behalf, logged with `punched_by`), single Punch In / Punch Out button toggle
- Rule: one open punch-in per staff per day; punch-out button only enabled if a punch-in exists without a matching punch-out.

2.7 Customer Profile / CRM Screen

- Read view: visit history (from completed Bookings/Invoices), active packages, preferences, tags
- Rule: tags can be manual (owner-assigned, e.g. "VIP") or system-derived (e.g. "inactive 60+ days" computed nightly) — keep these as two visibly different tag types in UI so staff don't manually remove a system tag by mistake.

2.8 Campaign / WhatsApp Automation Screen (Owner/Manager only)

- Fields: select trigger type, select/edit template, preview audience count before sending,

schedule or send now

- Rule: this screen writes to a queue table, actual sending happens via a scheduled job hitting WhatsApp Business API — do not send synchronously from this screen, campaigns can be hundreds of messages.

2.9 User Management Screen (Owner/CEO only — see RBAC Section 6)

- Fields: Create user (name, phone, role dropdown), assign/override individual menu permissions, activate/deactivate user
- This is the screen that satisfies "CEO should be able to assign menus to user / create user."

2.10 Reports Screens (per report type listed in main guide)

- Standard filter bar: date range, staff filter, service-category filter
- Export to Excel/PDF on all report screens

4. Commission Calculation — Configurable Engine

Client's actual rule (build this as the default, but make it fully configurable via admin UI, not hardcoded):

If a staff member's total sales in a month reach **5x their salary**, they get a bonus of **10% of their salary**.

This is a *threshold-bonus* model, not a per-transaction commission — different shape from a flat % per service. Structure `CommissionSlab.rules_json` to support this as one configurable type, alongside others, so the owner can change the multiplier/bonus % or switch models entirely without a code change:

Type: Threshold Bonus (client's current rule)

```
{
  "type": "threshold_bonus",
  "sales_multiplier_of_salary": 5,
  "bonus_pct_of_salary": 10
}
```

Calculation: at month-end payroll run, sum `InvoiceLineItem` amounts where `staff_id = X` for the month → if `total_sales >= staff.base_salary * sales_multiplier_of_salary`, add `staff.base_salary * (bonus_pct_of_salary / 100)` to that month's payroll as bonus.

Other types to support in the same engine (owner can pick per staff or globally, via Settings screen — not developer-configured):

Type: Flat % per service category

```
{ "type": "flat_pct", "default_pct": 10, "category_overrides": { "spa": 15,
"bridal": 20 } }
```

Type: Tiered by staff's monthly revenue generated

```
{ "type": "tiered", "tiers": [  
  { "min_revenue": 0, "max_revenue": 50000, "pct": 8 },  
  { "min_revenue": 50001, "max_revenue": 100000, "pct": 12 },  
  { "min_revenue": 100001, "max_revenue": null, "pct": 15 }  
]}
```

Type: Flat % split by item type (service vs retail product)

```
{ "type": "flat_by_item_type", "service_pct": 10, "product_pct": 5 }
```

Dev instructions:

- Build a **Commission Settings screen** (Owner/CEO only) where the rule type and its parameters (multiplier, bonus %, tier breakpoints, etc.) are editable — this must be a UI-driven config, not something only a developer can change.
- Commission calculator is a single function `calculateCommission(staff_id, period)` that reads the staff's assigned `CommissionSlab.rules_json` and resolves the amount — one function, multiple strategies, selected by `type`.
- Slab changes must NOT retroactively recalculate already-finalized payroll runs — only apply to periods calculated after the change.
- Since the client's actual rule is monthly-threshold-based (not per-line-item), commission for this rule type is computed once at `PayrollRun` generation time, not per invoice like the other types — design the payroll job to handle both calculation timings (per-transaction accrual for flat/tiered types, month-end aggregate check for threshold type).

5. Package Math — Worked Example (client mentioned "buy 2 get 1 free ₹999")

Confirm with client which interpretation is correct — build UI to support this generically:

Interpretation 1 — Single-service bundle

- ₹999 buys 3 credits of ONE specific service (e.g. "3 haircuts for ₹999" — normally ₹1200)
- `PackageMaster.credit_count = 3, included_services = [service_id: haircut]`
- Redemption: each visit consumes 1 credit until 0 remain, then status → exhausted

Interpretation 2 — Any-service credit pool

- ₹999 buys 3 "visit credits" usable against ANY service up to a price cap
- `included_services = []` (empty = any), plus `discount_logic_json` carries a max-value-per-credit cap

```
{ "credit_value_cap": 400, "overage_rule": "customer_pays_difference" }
```

Interpretation 3 — Straight discount, not credits

- "Buy 2 get 1 free" is really "33% off if you book 3 sessions today" — a discount rule, not a stored package at all
- This wouldn't even need `CustomerPackage` — just a promo/discount rule applied at billing

Dev instruction: do NOT assume Interpretation 1. Ask the client for one real example with actual rupee numbers and service names before building the redemption logic — this is the single most error-prone part of the whole system if guessed wrong, since it touches billing, CRM expiry alerts, and reporting simultaneously.

6. RBAC — Full Specification

5.1 Roles (seed data — confirm naming with client but structure stays as below)

Role	Who	Access Level
Owner/CEO	Client herself	Full access to every module + User Management (create users, assign roles, override individual permissions)
Manager	Senior staff/second-in-command (if exists)	Full operational access (Bookings, Billing, CRM, Inventory, Reports) — NO User Management, no Payroll edit (view only optional)
Receptionist/Front Desk	Day-to-day operator	Bookings, Billing/POS, Customer lookup — no Reports, no Settings, no Payroll, no CRM campaign screen
Stylist/Staff	Service staff	Own attendance punch, own booking calendar (view only), own commission/earnings view — nothing else
Accountant (optional, add if client wants)	External/part-time accountant	Reports (financial only), Payroll (view/export) — no Bookings/Billing edit access

5.2 Permission Model

- Every module has granular actions: `view`, `create`, `edit`, `delete`, `approve` (approve used sparingly — e.g. stock-override, discount-override).
- `RolePermission` table maps default access per role — this is what "different menu for different role" actually means: the left-nav renders only modules where the logged-in user

has at least `view` on any action within that module.

- `UserMenuOverride` allows CEO to grant/revoke a SPECIFIC permission to a SPECIFIC user beyond their role default — e.g. "give this one receptionist view access to Reports" without making her a full Manager. This must be additive/subtractive per user, not a role change.

5.3 The "Single Link, Different Menu" Requirement — Implementation Note

- One login URL, one login form, for everyone.
- On successful auth, backend resolves: `user.role` → `RolePermission` + any `UserMenuOverride` rows → final permission set for this session.
- Frontend fetches this permission set once at login (or via a `/me/permissions` endpoint) and renders the left-nav dynamically — **do not hardcode role checks scattered across the frontend** (if `role === 'owner'` in 15 places is a maintenance trap). Build one central `hasPermission(module, action)` helper the whole frontend calls.
- Every API endpoint must ALSO check permissions server-side, independent of frontend menu rendering — the frontend hiding a button is not security, it's UX. A Receptionist hitting the Reports API directly (via browser devtools) must get a 403, not just a hidden menu item.

5.4 User Creation Flow (Owner/CEO capability)

- Owner/CEO → User Management screen → "Create User": name, phone, role (dropdown from Role table)
- On creation: system auto-generates temp password / sends WhatsApp/SMS invite with login link (reuse WhatsApp API already in scope for CRM — no separate email system needed)
- Owner can later open any user's profile → toggle individual permission overrides, deactivate user (soft delete — never hard-delete a user who has historical Bookings/Invoices/Commission tied to them, just set `is_active = false` and block login)

5.5 Audit Trail (recommended, flag as Phase 1 if budget allows, else Phase 2)

- Log: who created/edited/deleted what and when, especially for: stock overrides, discount overrides, package redemptions, payroll finalization, user permission changes
 - Minimum viable version: a single `AuditLog(id, user_id, action, entity, entity_id, timestamp, details_json)` table written to on all sensitive actions — cheap to build now, expensive to retrofit later.
-

7. Tech Stack (Fixed)

- **Frontend:** React (.jsx) — matches existing Celeris ERP dashboard shell pattern (reference: AIMS platform UI)
 - **Backend:** Node.js/Express (.js)
 - **DB:** MongoDB (Mongoose) — per MERN standard. **Dev note:** several entities here are highly relational (commission entries ↔ invoice line items ↔ package credits ↔ payroll runs). Enforce referential integrity at the application layer since Mongo won't do it for you — use consistent `ObjectId` references per the schema in Section 2, and use Mongo multi-document transactions for any operation touching more than one collection at once (e.g. billing a package redemption must atomically decrement `CustomerPackage.credits_remaining`, write the `InvoiceLineItem`, and write the `CommissionEntry` — all or nothing).
 - **Auth:** JWT-based session, permission set resolved server-side per Section 6.3
 - **Mobile:** PWA (installable, add-to-homescreen) — not native/React Native. Build the React web app mobile-responsive from day one so the PWA wrap is trivial. This covers the owner-facing dashboard-on-phone use case; native app store presence would be a separately scoped future conversation.
 - **WhatsApp:** Meta WhatsApp Business API (Cloud API) — start the business verification/approval process immediately in parallel with dev, this has the longest external lead time of anything in this project
-